default scope order, as shown in figure 21.5. Finally, the filter with `Order = 1` executes.

By default, if a filter doesn't implement `IOrderedFilter`, it's assumed to have `Order = 0`. All the filters that ship as part of ASP.NET Core have `Order = 0`, so you can implement your own filters relative to these.

NOTE You can completely customize how the filter pipeline is built by customizing the MVC framework's *application model* conventions. These control everything about how controllers and Razor Pages are discovered, how they're added to the pipeline, and how filters are discovered. This is an advanced concept, that you won't often need, but it may occasionally come in handy. You can read about the MVC application model in the documentation at http://mng.bz/nWNa.

This chapter has provided a lot of background on the MVC filter pipeline, and we covered most of the technical details you need to use filters and create custom implementations for your own application. In chapter 22 you'll see some of the built-in filters provided by ASP.NET Core, as well as some practical examples of filters you might want to use in your own applications.

Summary